

TUGAS KECIL II — IF2211 STRATEGI ALGORITMA

3D Voxelization using Octree

Semester II 2025/2026

Made Branenda Jordhy — 13524026

Valentino Daniel Kusumo — 13524104

School of Electrical Engineering and Informatics

Institut Teknologi Bandung

March 25, 2026

Abstract



Contents

1	Introduction	3
2	Theoretical Background	3
2.1	Voxelization	3
2.2	Octree	4
2.3	Divide and Conquer	4
2.4	Separating Axis Theorem (SAT)	4
3	Divide and Conquer Algorithm	5
3.1	High-Level Steps	5
3.2	Pseudocode	5
3.3	Complexity Analysis	5
4	Implementation	5
4.1	System Architecture	5
4.2	Program Flow	5
4.3	Key Implementation Details	5
5	Results and Analysis	5
5.1	Test Case 1	5
5.2	Test Case 2	5
5.3	Test Case 3	5
5.4	Test Case 4	5
5.5	Test Case 5	5
5.6	Performance Summary	5
6	Bonus Features	5
6.1	Concurrency	5
6.2	Interactive 3D Viewer	5
7	Conclusion	5
	Repository	6

1 Introduction

Dalam *computer graphic*, objek 3D itu umumnya disimpan sebagai *polygon mesh*. Format ini fleksibel dan ringan tetapi tidak selalu cocok untuk kebutuhan tertentu. Seperti yang kita tau, *grid based physics simulations*, *3D printing*, hingga game seperti Minecraft, itu semuanya membutuhkan representasi volumetrik yang lebih terstruktur. Di sinilah proses *voxelization* digunakan yaitu mengubah *polygon mesh* menjadi kubus kecil seragam yang disebut *voxel*.

Problem utamanya adalah kita diberikan sebuah file `.obj` berisi *polygon mesh* dan program harus menghasilkan `.obj` baru yang mana seluruhnya tersusun dari voxel yang seragam yang merepresentasikan objek aslinya. Namun, ada obstacle yang terletak pada scale, pada depth octree yang tinggi, jumlah candidate voxel cenderung tumbuh secara eksponensial (8^N pada depth N).

Lalu bagaimana caranya kita develop? Ya, dengan menggunakan paradigma *Divide and Conquer* melalui *Octree* (ya sesuai spesifikasi tucil...). Ruang 3D dibungkus oleh bounding box (AABB) berbentuk cubic lalu dipotong secara rekursif menjadi 8 sub-cubic pada setiap level. Di setiap step, kita melakukan uji perpotongan antara segitiga mesh dan kotak menggunakan *Separating Axis Theorem* (SAT). Kotak yang tidak bersinggungan dengan segitiga manapun langsung di-*pruned* sehingga hanya bagian yang relevan yang ditelusuri lebih dalam. Proses ini naturally membentuk paradigma *Divide and Conquer*, divide ruang, conquer setiap bagian independently, lalu terakhir tinggal combine.

Program yang kita develop di sini sepenuhnya dalam `C++`. Selain fungsionalitas utama (wajib), kami juga mengimplementasikan dua fitur bonus yaitu *concurrency* untuk paralelisasi rekursif pada octree menggunakan `std::thread` serta *interactive 3D viewer* dengan raylib yang memungkinkan user melakukan eksplorasi hasil voxelization.

2 Theoretical Background

2.1 Voxelization

Secara sederhana, *voxelization* adalah proses mengubah model 3D yang mulus (biasanya *polygon mesh*) menjadi kumpulan kotak - kotak kecil dalam grid 3D. Analogi sederhananya adalah jika dunia 2D memiliki *pixel*, maka dunia 3D memiliki *voxel* (*volumetric pixel*) sebagai blok bangunan utamanya. Setiap voxel merepresentasikan sebuah kubus kecil di ruang dan menyimpan informasi apakah area tersebut dianggap terisi atau kosong.

Dalam praktiknya, ada dua pendekatan utama. Pertama, *surface voxelization*, yang hanya menandai kotak - kotak pada bagian permukaan objeknya saja. Kedua, *solid voxelization*, yang benar - benar mengisi sampai ke bagian dalam objeknya. Pada tugas ini, pendekatan yang digunakan adalah *surface voxelization* karena target utama kita hanya struktur bentuk luarnya saja agar lebih efisien.

Mengapa metode ini penting? Karena banyak sistem lebih mudah bekerja pada data berbasis grid, misalnya untuk *3D printing*, visualisasi permainan, dan masih banyak contoh lainnya.

2.2 Octree

Octree adalah struktur data hierarkis untuk partisi ruang 3D. Satu node merepresentasikan sebuah *axis-aligned bounding box* (AABB) berbentuk kubus, lalu node tersebut dibagi menjadi 8 anak kubus yang ukurannya setengah pada masing-masing sumbu. Proses pembagian ini bisa dilakukan berulang sampai kedalaman tertentu.

Kelebihan utama Octree adalah kemampuannya melakukan *adaptive refinement*. Area ruang yang akan digunakan (misalnya dekat permukaan objek) dapat dibagi lebih dalam, sedangkan area kosong dapat dipangkas lebih awal. Hal ini jauh lebih efisien dibanding membuat seluruh grid *uniform* dari awal, terutama saat model hanya menempati sebagian kecil ruang.

Tentu ada *trade-off* antara kedalaman dan resolusi. Semakin dalam tingkat Octree-nya, voxel-nya akan semakin kecil dan detail objek akan semakin tajam. Namun, jumlah sel dapat meningkat secara drastis. Oleh karena itu, strategi *pruning* (pemangkasan) wajib untuk diimplementasikan agar komputasi tetap efisien.

2.3 Divide and Conquer

Paradigma *Divide and Conquer* bekerja melalui tiga tahap: *divide*, *conquer*, dan *combine*. Pertama, masalah besar dipecah menjadi submasalah yang lebih kecil. Kedua, setiap submasalah diselesaikan secara independen (*umumnya rekursif*). Ketiga, hasil submasalah digabung untuk membentuk solusi akhir.

Pada kasus voxelization berbasis Octree, pola ini muncul secara alami. Tahap *divide* dilakukan saat satu kubus dibagi menjadi 8 sub-kubus. Tahap *conquer* dilakukan dengan mengecek masing-masing sub-kubus: apakah perlu dibagi lagi, dipangkas, atau dijadikan voxel akhir. Tahap *combine* terjadi ketika seluruh hasil voxel dari subtree dikumpulkan menjadi satu kesatuan model yang utuh.

Keuntungan pendekatan ini adalah struktur rekursifnya bersih, modular, dan sangat cocok untuk dibuat paralel. Karena banyak subtree saling independen, proses *conquer* dapat dijalankan bersamaan menggunakan *thread* terpisah jika diperlukan (*fitur bonus*).

2.4 Separating Axis Theorem (SAT)

Inti dari voxelization permukaan adalah untuk mengecek *apakah sebuah segitiga mesh berpotongan dengan sebuah AABB?* Untuk itu, kita menggunakan *Separating Axis Theorem* (SAT).

Prinsip SAT menyatakan bahwa dua objek konveks (*himpunan titik atau bentuk geometri yang tidak memiliki lekukan ke dalam*) tidak berpotongan jika dan hanya jika ada setidaknya satu sumbu proyeksi yang memisahkan keduanya (*separating axis*). Jadi, untuk membuktikan dua objek berpotongan, kita cukup memastikan bahwa tidak ada sumbu pemisah.

Pada pengujian segitiga vs AABB, himpunan sumbu uji yang umum dipakai berjumlah 13, yaitu 3 sumbu normal AABB (sumbu x , y , z), 1 sumbu normal bidang segitiga, dan 9 sumbu hasil *cross product* antara 3 edge segitiga dengan 3 sumbu AABB. Jika pada semua sumbu tersebut proyeksi kedua objek saling overlap, maka segitiga dan AABB dinyatakan berpotongan.



3 Divide and Conquer Algorithm

3.1 High-Level Steps

3.2 Pseudocode

3.3 Complexity Analysis

4 Implementation

4.1 System Architecture

4.2 Program Flow

4.3 Key Implementation Details

5 Results and Analysis

5.1 Test Case 1

5.2 Test Case 2

5.3 Test Case 3

5.4 Test Case 4

5.5 Test Case 5

5.6 Performance Summary

6 Bonus Features

6.1 Concurrency

6.2 Interactive 3D Viewer

7 Conclusion



Repository

https://github.com/ethj0r/Tucil2_13524026_13524104